

ADTs, Linked Lists, Inverted Lists and Recursion

Four practical sessions • 45 minutes each

This practical is designed to help you understand the algorithms before worrying about Java syntax. Work in pairs. Each pair should contain at least one person comfortable with basic programming. Both members are expected to contribute.

What you will practise

- Representing a singly linked list with nodes and links.
- Adding, inserting and deleting nodes.
- Maintaining a list in sorted order.
- Analysing the cost of building a sorted linked list.
- Using a simple inverted list to associate words with document numbers.
- Writing algorithms in ordinary English and pseudocode before programming them.
- Tracing and writing simple recursive algorithms.

How the practical works

The first part deliberately uses paper rather than the IDE. Do not jump straight to Java. If you cannot explain what the links should do on paper, writing Java will not make the problem easier.

- Driver: operates the computer and types when programming begins.
- Navigator: explains the algorithm, checks the result and suggests tests.
- Swap roles regularly.

What you submit

Keep your written work. At the end of each session, submit or show the specified work. The purpose is to demonstrate that you were actively working through the problems—not to produce polished documentation.

Lab work contributes 5% of the course mark. The emphasis is on participation and useful work during the practical.

Session 1 — Linked lists on paper

No programming yet. Draw nodes and arrows. Discuss every answer with your partner.

1. Understand the representation

Consider:

head → [7 | next] → [12 | next] → [19 | null]

1. How many nodes are there?
2. What does head refer to?

3. What does the next field of the node containing 12 refer to?
4. How do you reach 19 starting from head?
5. What does an empty list look like?

2. Add at the beginning

Current list: head $\rightarrow$ 12 $\rightarrow$ 19 $\rightarrow$ null

A new node contains 7.

6. Draw the final list.
7. Which links have to change?
8. Which link must be changed first?
9. Describe the operation in your own words.

3. Insert in the middle

Current list: 7 $\rightarrow$ 12 $\rightarrow$ 19 $\rightarrow$ 25

Insert 18.

10. Draw the new node and the final list.
11. Which existing links change?
12. Which node(s) must you be able to reach before you can perform the insertion?
13. Write the insertion algorithm in ordinary English.

4. Delete

Current list: 7 $\rightarrow$ 12 $\rightarrow$ 18 $\rightarrow$ 19 $\rightarrow$ 25

Delete 18.

14. Draw the list after deletion.
15. Which link changes?
16. What information do you need in order to perform the deletion?
17. Write the deletion algorithm in ordinary English.

5. First pseudocode

Write pseudocode for:

- addFirst(x)
- insertAfter(previous, x)
- removeAfter(previous)

Do not worry about Java syntax. Concentrate on what must happen.

SESSION 1 CHECKPOINT: Submit/show your drawings, English descriptions and pseudocode.

Session 2 — Building and analysing a sorted linked list

This exercise connects the mechanics of linked-list insertion with algorithm analysis. Work it out on paper before programming.

1. Build a sorted list

We want to sort a collection of numbers using a singly linked list. Work out, schematically and on paper, how you could build the sorted list one number at a time. Draw the nodes and links as you insert several numbers. Do not write Java yet.

2. Finding the insertion point

Suppose a sorted list currently contains n elements. You want to insert one more number in the correct position so that the list remains sorted. How many list nodes might you have to visit to find the insertion point? Give an answer in terms of n . Consider whether the position of the new number matters.

3. Building the list

The insertion operation in Question 2 is repeated as you build the list: each new number is inserted into the correct position. Starting with an empty list, what is the expected total number of list-node visits when you insert n numbers? Show your reasoning.

4. A simple performance calculation

Suppose that visiting one list node takes 5 microseconds. Using your result from Question 3, estimate how many numbers can be inserted into the list within 30 seconds.

5. Interpretation

Explain in one or two sentences what your result tells you about building a sorted list with a singly linked list.

Hint: For Questions 2–4, make a small table for lists of 1, 2, 3, 4, ... elements. Look for the pattern before writing a formula.

6. Insert while keeping the list sorted

Start with: $3 \rightarrow 8 \rightarrow 14 \rightarrow 21 \rightarrow 31$

- Insert 17
- Insert 2
- Insert 40
- Insert 14

18. What special case occurs when inserting before the first node?
19. What special case occurs when the new value belongs at the end?
20. What should happen if duplicate values are allowed?
21. When can you stop searching for the insertion position?

7. Deleting from a sorted list

For the list $3 \rightarrow 8 \rightarrow 14 \rightarrow 21 \rightarrow 31$, work out how to delete 3, 21, 31 and 17.

Does keeping the list sorted make deletion easier, harder, or neither? Explain.

8. Simple inverted list

- Document 1: cat dog fish
- Document 2: dog bird
- Document 3: cat bird fish

Construct the inverted list:

- $\text{cat} \rightarrow 1 \rightarrow 3$

- dog $\rightarrow 1 \rightarrow 2$
- fish $\rightarrow 1 \rightarrow 3$
- bird $\rightarrow 2 \rightarrow 3$

Now suppose Document 4 contains dog. Show how the inverted list changes.

22. How would you find the entry for a word?

23. How would you add a document number to that word's list?

24. If document numbers must remain sorted, what operation from earlier can you reuse?

9. Plan before programming

Write pseudocode for:

- find(word)
- addWord(word, documentNumber)
- documentsContaining(word)

SESSION 2 CHECKPOINT: Submit/show the sorting analysis, sorted-list algorithms, and inverted-list design and pseudocode.

Session 3 — Implement and test

Now use the IDE. Translate the algorithms you developed on paper into Java.

A. Linked-list ADT

Implement a simple singly linked list supporting:

- isEmpty()
- addFirst(x)
- insertSorted(x)
- remove(x)
- contains(x)

Use the class structure supplied by your lecturer. Do not redesign the whole ADT.

B. Test systematically

- empty list
- one-element list
- insert at beginning
- insert in middle
- insert at end
- duplicate value
- remove first
- remove middle
- remove last
- remove a value not present

C. Simple inverted list

Implement the small inverted-list structure you designed in Session 2 using a small collection of 5–10 documents.

- add a word to the index
- add a document number to an existing word
- keep document numbers correct
- search for a word
- handle an unknown word sensibly

SESSION 3 CHECKPOINT: Submit/show your working program and test results.

Session 4 — Recursion

The goal is to understand the structure of a recursive algorithm, not merely to produce recursive code.

1. Trace a recursive function

Use the simple recursive function supplied by your lecturer. Draw the sequence of calls for $f(4)$.

25. What is the base case?
26. What happens in the recursive case?
27. Which call happens first?
28. In what order do the calls return?
29. What prevents the recursion continuing forever?

2. Explain recursion in English

For a function that processes a linked list recursively, describe what it should do when the list is empty and what it should do otherwise. Write this in ordinary English before writing code.

3. Pseudocode

Write pseudocode for a recursive linked-list traversal. Make the base case explicit.

4. Home assignment — recursive list traversal

Write a Java program containing a recursive function that traverses a singly linked list. The function should visit each node and perform a simple specified action, such as printing its value.

- what the base case is
- what the recursive case is
- what happens to the reference used to reach the next node
- why the function eventually stops

This home assignment contributes 5% of the course mark.

A final reminder

It is normal for the first version of an algorithm to be wrong. Draw it again. Trace it. Find the case or link you missed. The important skill in this practical is learning to reason about the data structure before relying on programming syntax.

Submission checklist

- Session 1: linked-list drawings, insertion/deletion explanations and pseudocode.
- Session 2: sorted-list construction, node-visit analysis, performance calculation, and inverted-list plan.

- Session 3: working Java program and evidence of testing.
- Session 4: recursion trace and pseudocode; home assignment submitted separately.